Chapter 3 Requirements Engineering

> *The hardest single part of building a software system is deciding precisely what to build*
>
> — Frederick Brooks

1 Introduction

Before we start to develop our software, it becomes quite essential for us to understand and document the exact requirement of the customer. Experienced members of the development team carry out this job. They are called as **system analysts**.

> 📖 DEFINITION 1: **Requirement**
>
> A requirement is a feature of the system or a description of something the system is capable of doing in order to fulfill the system's purpose.

2 Core Questionnaire

During the requirements gathering and analysis phase, a system analyst must bridge the **gap between the customer's vision and the technical specifications needed for development.** To gain a thorough and unambiguous understanding of the project, the analyst should systematically address the following foundational questions:

1. Problem Definition & Justification

- **What is the problem?**

Define the exact pain points, inefficiencies, or operational gaps in the current workflow that necessitate the development of the software.

- **Why is it important to solve the problem?**

  Establish the business value, urgency, and expected impact of addressing these issues on the organization or end-users.

2. Solution Exploration & Scope

- **What are the possible solutions to the problem?**

  Explore alternative approaches (e.g., custom development vs. commercial off-the-shelf software, or varying architectural paradigms) to ensure the most viable path is selected during the feasibility study.

3. Data Flow & Boundaries

- **What exactly are the data input to the system and what exactly are the data output by the system?**

  Identify the precise data sources, formats, and domains entering the system, as well as the expected outputs, reports, or results generated for the user. (This lays the groundwork for creating Data Flow Diagrams and Data Dictionaries).

4. Technical Complexities & Constraints

- **What are the likely complexities that might arise while solving the problem?**

  Pinpoint technical, architectural, or domain-specific risks (such as performance bottlenecks, concurrency hurdles, or algorithmic complexities) that need mitigation.

5. System Interfacing & Integration

- **If there are external software or hardware with which the developed software has to interface, then what exactly would the data interchange formats with the external system be?**

Specify the communication protocols, APIs, file structures (e.g., JSON, XML, binary formats), or hardware constraints required for seamless third-party integration.

6. Stakeholders & Business Value

- **Who is requesting the software?**

  Identify the sponsor or business stakeholder funding/initiating the project and defining its high-level objectives.

- **Who will use the software?**

  Characterize the primary and secondary end-users, distinguishing between different user personas and their distinct interaction requirements.

- **What is the benefit that the software will bring?**

  Summarize the qualitative and quantitative returns (e.g., time saved, error reduction, increased throughput, cost efficiency) delivered upon successful deployment.

> ⓘ **Note**
>
> After the analyst has understood the exact customer requirements, he proceeds to identify and resolve the various requirements problems. The most important requirements problems that the analyst has to identify and eliminate are the problems of anomalies, inconsistencies, and incompleteness. When the analyst detects any inconsistencies, anomalies or incompleteness in the gathered requirements, he resolves them by carrying out further discussions with the end users and the customers.

3 What is requirements engineering?

> 📖 DEFINITION 2: **Requirements Engineering**

> The process concerned with understanding the software system that the customer wants developed (and not how it should be developed).

3.1 Lifecycle of Requirements Engineering

Requirements engineering consists of the following processes where Each process builds systematically on the previous one to ensure that what gets built actually solves the right problem for the customer:

- **Inception:** Establishing the foundational understanding and setting the boundaries of what the system will touch in which the nature and scope of the system is defined.
- **Elicitation:** Interacting directly with stakeholders, users, and domain experts to pull out what they need the system to do.
- **Elaboration:** Taking those raw, often vague requirements and breaking them down into more detailed technical and operational models.
- **Negotiation:** Balancing competing stakeholder interests, trimming unrealistic feature lists, and resolving conflicting demands within budget and time constraints.
- **Specification:** Capturing the refined agreements in a formal artifact—typically a Software Requirements Specification (SRS) document or a structured model.
- **Validation:** Reviewing and auditing the documented requirements to guarantee they are accurate, feasible, unambiguous, and complete before heavy design begins.
- **Management:** Controlling how requirements evolve, handling change requests, and maintaining traceability as the project progresses through its lifecycle.

> **Note**
>
> These steps are overlapping for a variety of reasons. You should be able to notice that some of them, such as management, must occur throughout the communication and modelling activities. Negotiation will also occur at each of the various requirements engineering steps. More importantly, a process model which understands that requirements may be (initially) poorly understood, and that they may change through the project's lifetime, will also iteratively collect and detail the use cases (consider the unified process model from the previous chapter). This iterative process will forcefully overlap all of these steps

4 Types of software requirements

Software requirements are often categorized into these overlapping dimensions:

1. **By Nature:** Functional vs. Non-Functional.
2. **By Viewpoint / Abstraction:** User Requirements (what the user sees/wants) vs. System Requirements (how the system is specified technically).
3. **By Lifecycle Priority:** Mandatory/Essential (must-have) vs. Desirable (nice-to-have).

4.1 Functional versus Non-Functional Requirements

4.1.1 Functional requirements (FRs)

Functional requirements (FRs) define the fundamental behaviors, features, and business logic of a software system. They explicitly state what the system *should do*, how it must react to specific inputs, and how it should behave under predictable operational scenarios. In essence, functional requirements map directly to the transformations applied to input data to generate the expected output data structures:

$$\text{Inputs} \xrightarrow{\text{Functional Logic}} \text{Processed Data/Outputs}$$

If a system fails to fulfill a functional requirement, the product is fundamentally broken or incomplete with respect to its core purpose.

> 📝 **EXAMPLE 1: Online Banking System**
>
> Consider an online banking platform. The functional requirements dictate the core transactional capabilities available to both customer and administrative actors:
>
> 1. **Account Balance Reporting:** The system must retrieve and display the current available balance for a user's checking account within 1.5 seconds of a successful authentication request.
> 2. **Fund Transfers:** The system must allow authenticated users to transfer funds between internal accounts or execute wire transfers to external bank accounts, provided the source account contains sufficient funds.
> 3. **Debit Card Management:** The system must provide a dedicated interface allowing users to temporarily lock or permanently cancel their debit cards.

4.1.1.1 Characteristics of Well-Formed Functional Requirements

- **Determinism:** Given a specific input state S_0 and a trigger event E, the system must transition to a predictable output state S_1.
- **Completeness:** Every user interaction, data processing rule, and system calculation must be accounted for without ambiguous edge cases.
- **Traceability:** Functional requirements must link directly to high-level user stories, use case models, and subsequent system-level design artifacts (such as class diagrams or data-flow diagrams).

4.1.2 Non-Functional Requirements

Non-functional requirements (NFRs)—frequently referred to as **quality attributes**,**constraints**, or **system qualities**—define *how well* the system performs its

functions. Rather than describing specific features, NFRs establish the operational, environmental, and architectural boundaries under which the system must operate.

While a functional requirement dictates that the system *must* process a bank transfer, a non-functional requirement dictates that the transfer interface *must* maintain 99.99% availability and encrypt all transaction payloads using AES-256 standards.

> **Note**
>
> Functional requirements are frequently specified in **natural language** (e.g., in English). Conversely, non-functional requirements are specified using **metrics.**

4.1.2.1 Quantifying Non-Functional Requirements

Because non-functional requirements dictate *how well* a system operates rather than what features it provides, they are notoriously susceptible to vague and ambiguous phrasing. Statements like "*the system will have high availability*" or "*the response time will be quick*" are untestable because "high" and "quick" mean different things to different stakeholders.

To prevent ambiguity during design, implementation, and acceptance testing, professional software specifications mandate that **non-functional requirements must be defined using objective metrics**.

Table 2: *Standard Non-Functional Metrics Matrix*

Non-Functional Requirement	Description & Focus	Standard Engineering Metric	Example Quantification
Performance	Speed, latency, throughput, and resource responsiveness.	Transactions per second (TPS), response time, latency, throughput.	99% of transactions executed in any 5-minute window must have a response time $\leq 1.0\ \text{second}$.
Space / Storage	Efficiency in utilizing hardware storage and memory capacity.	Storage usage, RAM consumption, cache footprint.	The mobile client app package size must not exceed $50\ \text{MB}$, and memory usage during idle state must remain under $100\ \text{MB}$.
Reliability	System dependability and continuity of service.	Uptime percentage, Mean Time Between Failures (MTBF).	System uptime of 99.99% (allowing a maximum cumulative downtime of $\approx 52.6\ \text{minutes/year}$).
Robustness / Fault Tolerance	Ability to recover from unexpected failures or erroneous inputs.	Mean Time To Recover (MTTR), risk of data loss after crash.	System must automatically recover from database node failures within $30\ \text{seconds}$ with zero data loss.

Non-Functional Requirement	Description & Focus	Standard Engineering Metric	Example Quantification
Usability	Ease of learning, efficiency, and user satisfaction.	User training time, task completion rate, error frequency.	A novice user must be able to complete checkout within $3\ \text{minutes}$ after a maximum of $15\ \text{minutes}$ of training.
Portability	Ease of adapting software across different platforms or environments.	Percentage of portable lines of code, environment dependencies.	100% of core business logic must be platform-agnostic and compile natively on Linux, macOS, and Windows.

4.2 User Requirements versus System Requirements

Beyond classifying requirements by their intrinsic nature (functional vs. non-functional), requirements engineering categorizes specifications by **viewpoint and abstraction**. Originally popularized in software engineering literature by Ian Sommerville, this perspective differentiates between the high-level needs of the domain stakeholders and the rigorous technical specifications required by developers and architects.

4.2.1 User Requirements

User requirements are high-level statements written in natural language, accompanied by intuitive diagrams or operational models, that describe what the user expects the system to accomplish. They focus entirely on the **problem domain** rather than the technological solution.

User requirements are typically authored by business analysts, product owners, or end-users themselves. They should be accessible to non-technical stakeholders, avoiding internal system architecture jargon, database schemas, or low-level interface protocols.

Characteristics & Best Practices

- **Audience:** Designed for clients, managers, domain experts, and end-users.
- **Language:** Natural language combined with sketches, use case summaries, or user stories.
- **Focus:** States *what* services the system is expected to provide and under what operational constraints the business operates, without prescribing how the software will implement them internally.

4.2.2 System Requirements

System requirements—often referred to in engineering specifications as the **Functional System Specification (FSS)**—are detailed, precise, and structured descriptions of the software's behavior, internal data structures, and operational constraints. They bridge the gap between the business problem and the software solution, serving as the definitive contract between the customer and the development team.

While a user requirement might state that a user needs to securely transfer funds, the corresponding system requirements specify the exact cryptographic protocols, API endpoints, database transaction isolation levels, and error-handling routines required to make that transfer happen.

Characteristics & Best Practices

- **Audience:** Designed for software architects, developers, testers, and system administrators.
- **Language:** Structured natural language, formal specification languages, sequence diagrams, class diagrams, and state transition models.
- **Focus:** States *how* the system will be built, detailing technical components, interfaces, algorithms, and compliance standards.

4.2.3 The Expansion Relationship: From User to System

A fundamental principle of requirements engineering is that **a single user requirement frequently expands into a set of multiple system requirements**.

To illustrate this expansion, consider the following mapping from an online banking domain:

Requirement Level	Specification Statement	Description & Focus
User Requirement (UR_1)	*"The system must allow registered customers to transfer funds securely from their checking account to an external bank account via wire transfer."*	**Problem-Oriented:** Written in natural language for the bank customer and business analyst. Focuses on business utility and user goals.
System Requirement ($SR_{1.1}$)	*"The transfer module must validate that the source account balance $\geq$ requested transfer amount plus applicable wire fees prior to initiating the transaction."*	**Solution-Oriented:** Specifies internal business logic constraints and validation rules.
System Requirement ($SR_{1.2}$)	*"All external wire transactions must communicate with the SWIFT gateway using TLS 1.3 encryption and mutual certificate authentication."*	**Technical Constraint:** Specifies security protocols, encryption standards, and interface communication channels.
System Requirement ($SR_{1.3}$)	*"If an external banking gateway timeout (≥ 5.0 seconds) occurs, the transaction must roll back atomically and display error code* `ERR_GW_TIMEOUT` *to the user."*	**Fault Tolerance:** Specifies technical error handling, state rollback rules, and exception codes for developers and testers.

4.3 Examples

📝 EXAMPLE 2:

Draft Requirement: "*The portal system should be fast, secure, and let students register for courses while keeping all student data accurate and consistent across the campus network.*"

✅ Show Solution

Is it Functional or Non-Functional?

- It is a **hybrid (mixed) requirement**.
- "*let students register for courses*" is a **Functional Requirement (FR)** because it describes a specific system feature/behavior.
- "*fast, secure, and keeping all student data accurate and consistent*" describes performance, security, and data integrity attributes, which are **Non-Functional Requirements (NFRs)**.

Is it complete and correct?

- **No**, it is incomplete and poorly structured.
- **Missing Details:** It fails to specify *which* courses can be registered, what validation rules apply (e.g., prerequisite checks), what security standard or encryption protocol is required, and what constitutes "fast" response times.

Can a QA engineer test this as written?

No. Words like "fast" and "secure" are subjective and qualitative . A testing team cannot write an objective pass/fail test for whether a system is "fast" unless concrete metrics (like milliseconds of latency or transactions per second) are provided .

Refinement & Rewriting

To make this professional, precise, and **verifiable**, the single compound sentence must be split into distinct, measurable requirements:

1. **Functional Requirement (FR):**
    - "*The portal system shall allow authenticated students to add and drop courses during official registration windows, validating that prerequisite courses have been successfully completed prior to enrollment.*"
2. **Non-Functional Requirement - Performance (NFR):**
    - "*The portal system shall ensure that* 99% *of course registration requests are processed with a maximum response time of* ≤ 1.5 seconds *during peak load.*"
3. **Non-Functional Requirement - Security & Privacy (NFR):**
    - "*All student data transmitted across the campus network must be encrypted using TLS 1.3, and passwords must be hashed using bcrypt.*"

📝 **EXAMPLE 3:**

Imagine your team is developing the **AAUP Digital Course Evaluation and Feedback Module**. A junior developer wrote the following non-functional requirement for the system:

> **Draft Requirement:** "*The student course evaluation module must be extremely reliable and handle heavy traffic whenever final semester grades are published.*"

- **Identify the Flaw:** Why is this requirement impossible to test or verify objectively by a Quality Assurance (QA) engineer?
- **Your Turn:** How would you rewrite and quantify this requirement into a **verifiable** non-functional requirement (using proper metrics like uptime, availability, or response times) so that it can be objectively tested?

✅ Show Solution

This requirement is **completely qualitative, vague, and untestable**.

- Words like **"extremely reliable"** and **"handle heavy traffic"** are subjective.
- A Quality Assurance (QA) engineer cannot write a test case for "extremely reliable" because there is no numerical threshold. If the system crashes under 10,000 users, does that count as handling heavy traffic? Without explicit metrics, nobody can prove whether the requirement has passed or failed.

2. How to Rewrite and Quantify It (To Make it Verifiable)

To make this non-functional requirement **verifiable**, it must be rewritten using objective metrics for **reliability** and **performance/throughput** (following standard guidelines found in software engineering textbooks like):

- **Rewritten Verifiable Requirement - Reliability:**

> *"The student course evaluation module shall maintain an availability (uptime) of* 99.9% *during grade release week, with a Mean Time To Recover (MTTR) of less than* 15 minutes *in the event of any unexpected failure."*

- **Rewritten Verifiable Requirement - Performance & Capacity:**

> *"The module shall support a minimum throughput of* 500 $\text{transactions per second}$ (TPS) *and maintain a maximum response time of* ≤ 1.5 seconds *during peak final grade publishing periods."*

5 The Requirements Specification Document (RSD)

Once requirements are elicited from stakeholders, they undergo a rigorous process of analysis, refinement, and organization. In traditional or plan-driven software engineering projects (such as those following waterfall-based lifecycles), these requirements are formalized into a comprehensive artifact known as the **Requirements Specification Document (RSD)** or **Software Requirements Specification (SRS)**.

The RSD serves as the definitive technical blueprint and contract between the customer, project managers, and the development team, establishing precisely what the software must accomplish prior to architectural design and coding.

> **Agile Methodologies**
>
> In Agile development, requirements are documented using **user stories** rather than relying on massive upfront requirement documents, agile frameworks favor lightweight, iterative backlogs that evolve continuously in close collaboration with the customer. However, in plan-driven projects, enterprise contracts, or safety-critical domains, a formal Requirements Specification Document remains necessary.

5.1 IEEE requirements standard

To bring discipline and uniformity to software documentation in traditional models, the **IEEE 830 Standard** was introduced in the 1990s. Although agile teams favor flexibility, the structural layout established by IEEE 830 provides a robust template for large-scale systems where comprehensive documentation is contractual and mandatory.

An IEEE 830-compliant document systematically organizes requirements into clearly defined sections to eliminate ambiguities.



IEEE Guide to Software Requirements Specifications OPEN PDF

IEEE Recommended Practice for Software Requirements Specifications
https://standards.ieee.org/ieee/830/1222/
OPEN LINK

5.1.1 IEEE 830 Standard SRS Structure & Main Headings

- **Software Requirements Specification (IEEE 830)**
  - **1. Introduction**
    - 1.1 Purpose
    - 1.2 Document Conventions
    - 1.3 Intended Audience and Reading Suggestions
    - 1.4 Product Scope
    - 1.5 References
  - **2. Overall Description**
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design and Implementation Constraints
    - 2.6 Assumptions and Dependencies
  - **3. Specific Requirements (The Core Technical Section)**
    - **3.1 External Interface Requirements**
      - 3.1.1 User Interfaces
      - 3.1.2 Hardware Interfaces
      - 3.1.3 Software Interfaces
      - 3.1.4 Communications Interfaces
    - 3.2 Functional Requirements
    - 3.3 Performance Requirements
    - 3.4 Logical Database Requirements
    - 3.5 Design Constraints
    - 3.6 Software System Attributes (Reliability, Availability, Security)
    - 3.7 Other Requirements

- Appendices
- Index

Detailed Description of Main Sections

1. Introduction

- **Purpose:** Defines the goals and objectives of the software product being specified.
- **Document Conventions:** Explains any formatting standards, typography, or notation rules utilized throughout the document.
- **Intended Audience:** Identifies who will read the document (e.g., developers, project managers, clients, testers) and suggests reading paths.
- **Product Scope:** Outlines what the software will and will not do, linking it to broader organizational or business goals.
- **References:** Lists related documents, standards, or source materials.

2. Overall Description

- **Product Perspective:** Contextualizes the software within its environment, showing whether it is a standalone product, a component of a larger system, or a replacement for legacy software.
- **Product Functions:** Summarizes the major capabilities and high-level features the software will execute.
- **User Classes and Characteristics:** Identifies the different types of users (e.g., administrators, regular users, guests) and details their technical expertise and frequency of use.
- **Operating Environment:** Describes the hardware, operating system, and network infrastructure where the software will run.
- **Constraints & Assumptions:** Outlines any regulatory limitations, hardware restrictions, or underlying assumptions that could affect the project.

3. Specific Requirements

This is the largest, most rigorous section containing all the detailed technical specifications required by developers and testers:

- **External Interface Requirements:** Detailed specifications for how the system connects with humans (UI), hardware devices, other software, and communication protocols.
- **Functional Requirements:** The core behavioral features, input/output processing rules, and computational logic of the system.
- **Performance Requirements:** Quantified metrics regarding speed, throughput, response time, and capacity.
- **Database Requirements:** Data structures, logical storage schemas, and retention policies.
- **System Attributes (Non-Functional):** Security, privacy, maintainability, availability, and reliability metrics.

Appendices & Index

- Includes supplementary material such as analysis models, UI wireframes, rough cost estimations, or glossaries of domain-specific terminology.

5.2 AAUP Integrated Student & Campus Portal Module

5.2.1 Purpose & Scope

- **Purpose:** To specify the complete functional requirements, user workflows, and technical performance constraints for the new **AAUP Campus Portal**, a centralized web and mobile platform designed to streamline academic registration, tuition tracking, course management, and campus services for students and faculty.
- **Scope:** The system encompasses a responsive web portal and mobile application interface, backend database integration with the university's central Student

Information System (SIS), and secure APIs for external campus services (such as parking access and library checkouts).

5.2.2 Definitions, Acronyms, and Abbreviations

- **SIS:** Student Information System.
- **GPA:** Grade Point Average.
- **API:** Application Programming Interface.
- **SSO:** Single Sign-On (institutional authentication).
- **RFID:** Radio Frequency Identification.
- **MFA:** Multi-Factor Authentication.

5.2.3 Overall Description

5.2.3.1 Product Perspective & Environment

The AAUP Portal operates as a cloud-hosted, multi-tier web application. It integrates directly with existing legacy databases on campus, authenticates users via institutional Microsoft/Exchange credentials, and serves thousands of concurrent users during peak academic periods (such as course registration week). Furthermore, it interfaces with campus entry RFID gate scanners to sync student parking access and attendance logs in real time.

5.2.3.2 Product Functions (Summary of Capabilities)

- **Academic Management:** Course registration, grade reporting, and real-time degree audit tracking.
- **Financial Ledger:** Tuition fee calculations, online payment gateway integration, and financial clearance tracking.
- **Communication Hub:** Official announcements from faculty members, department heads, and administration.

- **Campus Integration Services:** Real-time parking lot availability checks and digital campus service hooks.

5.2.3.3 User Classes and Characteristics

- **Undergraduate & Graduate Students:** High-volume users accessing the portal daily for schedules, grades, and announcements. They require an intuitive, fast mobile-first interface.
- **Faculty Members & Instructors:** Academic staff responsible for posting grades, uploading course materials (lecture notes and syllabi), and managing class rosters via desktop workstations or mobile screens.
- **Administrative Staff / Admissions:** Users with elevated privileges to manage student records, resolve registration blocks, and generate institutional accreditation reports.

5.2.3.4 Operating Environment & Constraints

- The portal must be fully accessible across modern web browsers (Chrome, Firefox, Safari, Edge) and mobile operating systems (iOS and Android).
- The system must handle high traffic surges during the first week of semester add/drop periods without performance degradation.
- System maintenance windows must be restricted to designated off-peak weekend hours to avoid disrupting active academic periods.

5.2.4 Specific Requirements

5.2.4.1 External Interface Requirements

- **User Interfaces (UI):** Responsive design adapting automatically to smartphone screens, tablets, and desktop workstations used in faculty offices, adhering strictly to AAUP institutional branding guidelines.
- **Hardware Interfaces:** The backend must interface with passive RFID gate readers at campus entry points to read student/staff parking credentials and log entry

timestamps.

- **Software Interfaces:** Integration with the university's Microsoft Exchange/Active Directory for Single Sign-On (SSO) authentication, the email server for automated notifications, and the accounting department's payment gateway for online tuition processing.
- **Communication Interfaces:** All data exchanges between clients, mobile applications, and cloud backend servers must communicate via HTTPS using TLS 1.3 encryption protocols.

5.2.4.2 Functional Requirements (Grouped by Feature)

- **FR-01 (Course Registration):** The system must allow authenticated students to add or drop elective and core courses during official registration windows, validating prerequisites (e.g., ensuring a student has passed *Introduction to Database Systems* before registering for advanced courses) in real-time.
- **FR-02 (Grade Posting):** Instructors must be able to input, verify, and submit final semester grades for class rosters, with the system permanently locking submissions after the official academic calendar deadline passes.
- **FR-03 (Financial Clearance Check):** The system must automatically check a student's tuition balance ledger and block course registration if outstanding dues exceed the financial threshold set by the university administration.
- **FR-04 (Parking Gate Sync):** When a registered vehicle passes an entry RFID scanner, the system must update lot availability metrics and verify active student status instantaneously.

5.2.4.3 Non-Functional Requirements (Quantified Metrics)

- **Performance:** Page load times for student grade reports and course schedules must not exceed $1.5\ \text{seconds}$ under normal operational load, with maximum query latencies under $1.0\ \text{second}$ for 99% of transactions during peak hours.
- **Availability:** The portal must maintain an uptime of 99.9% throughout the academic semester.

- **Security & Privacy:** All user sessions must be encrypted using TLS 1.3, passwords must comply with institutional security policies, and accounts must be protected via multi-factor authentication (MFA).

6 User Stories

User Stories are popular, lightweight instruments used primarily in **Agile methods** to define software requirements from an end-user's perspective.

Instead of writing massive, traditional waterfall documents that take months to complete, Agile teams use user stories to capture requirements concisely, keeping the focus on *who* needs the feature,*what* they want, and *why* they need it.

> **Note**
>
> It is also recommended to define the key users who will interact with the system before writing stories.

6.1 The Three Cs of a User Story

As defined by Ron Jeffries, a user story consists of three vital parts:

1. **Card:** A physical or digital index card containing a short, natural-language sentence written by the customer or product owner describing a feature.
2. **Conversations:** Verbal discussions between the developers, testers, and customers to flesh out the detailed specifics, edge cases, and acceptance criteria.
3. **Confirmation:** The acceptance tests that prove the user story has been implemented successfully.

6.2 User Stories properties

- **Independent:** It should be possible to implement any two stories in any order. Ideally, there should be no dependencies between stories.
- **Negotiable:** As mentioned earlier, stories (the card) are invitations for conversations between customers and developers during a sprint. Both parties should be open to changing and adapting their opinions as a result of these discussions. Developers should be open to implementing details not expressed on or that do not fit on the story cards. Customers should also consider technical arguments from developers, such as those regarding the complexity of implementing some aspects of the story as initially planned.
- **Valuable:** Stories should add value to the customers' business. Stories are proposed, written, and ranked by the customers according to the value they add to their business. Consequently, the idea of a technical story—such as the system must be implemented in JavaScript, using React and Node.js—is inappropriate.
- **Estimable:** It should be possible to estimate the size of a story, i.e., to define the effort needed to implement it. Typically, this requires the story to be small, as we will discuss later. Estimation also becomes easier when the developers have experience in the system's domain.
- **Small:** Complex and large stories—also known as **epics**—can exist, but they should be placed at the bottom of the backlog, indicating they will not be implemented soon. On the other hand, stories at the top of the backlog should be short and small to facilitate understanding and estimation. For a one-month sprint, it should be possible to implement any story in less than one week, for example.
- **Testable:** Stories should have clear acceptance tests. For example, the customer can pay by credit card is testable, provided we know the kind of credit card that will be used. Conversely, a story like a customer should not have to wait too long to have their purchase confirmed is not testable. This is because it lacks a clear acceptance test.

6.3 Standard Format of a User Story

The most common template used to write a user story is:

> **As a** [Type of User], **I want** [some goal or feature], **So that** [some business value or benefit].

🎬 SCENARIO 1: Library Management System

In this section, we provide examples of user stories for a library management system. These stories are associated with three user types: students, instructors, and library staff.

First, we present stories suggested by students (see below). Any library user can perform the operations described in these stories. Note that the stories are brief and do not elaborate on how each operation should be implemented. For example, one of the stories states that students should be able to search for books. However, many details are omitted, including search criteria, available filters, limits on the number of search results, and the layout of search and results pages. Nonetheless, we should remember that a story is essentially a commitment: the customer representative ensures they will be available to clarify these details with developers during the sprint in which the story is implemented. When working with user stories, this verbal interaction between developers and the customer representative is crucial to successful requirements specification and implementation.

> As a student, I want to borrow books.
>
> As a student, I want to return books I have borrowed.
>
> As a student, I want to renew my book loans.
>
> As a student, I want to search for books.
>
> As a student, I want to reserve books that are currently on loan.
>
> As a student, I want to receive notifications about new acquisitions.

Now, we present the stories suggested by the instructors:

> As an instructor, I want to borrow books for an extended period of time.
>
> As an instructor, I want to recommend books for acquisition.
>
> As an instructor, I want to donate books to the library.
>
> As an instructor, I want to return books to other campus libraries.

Although these stories originate from instructors, this does not mean they are exclusive to this user group. For example, during the sprint, the customer representative (or Product Owner) may consider making the donation feature available to all users.

The last story proposed by instructors—allowing books to be returned to any university library—can be classified as an **epic**, that is, a complex story. This story refers to a scenario where an instructor borrows a book from the central library but wants to return it to a departmental library, or vice versa. Implementing this story is more complex as it requires integrating different library systems and having staff members transport the books back to their original locations.

Finally, we share the stories proposed by the library staff members, typically concerning library organization and ensuring its seamless operation:

> As a staff member, I want to register new users.
>
> As a staff member, I want to add new books to the system.
>
> As a staff member, I want to remove damaged books from the system.
>
> As a staff member, I want to access statistics about the collection.
>
> As a staff member, I want the system to send reminders to users with overdue books.

> As a staff member, I want the system to apply fines for late book returns.

To confirm the implementation of the search story, the customer representative defined the following test scenarios:

> Search for books using an ISBN.
>
> Search for books using an author's name.
>
> Search for books using a title.
>
> Search for books added to the library on or after a specific date.

The correct implementation of these searches will be demonstrated during the Sprint Review meeting, assuming the team is using Scrum.

As previously mentioned, acceptance tests are specified by the customer representative (or Product Owner). This practice helps prevent a scenario known as **gold plating**. In Requirements Engineering, this term describes the situation where developers independently decide to elaborate on certain stories—or requirements, more generally—without the customer's input. Metaphorically, developers are embellishing stories with layers of gold, which does not generate value for users.

6.4 The Lifecycle of User Stories in Agile

- **Created & Added Directly to the Product Backlog:**
  - When user stories (such as *"As a student, I want to borrow books"*) are first elicited and written during workshops or brainstorming sessions,**they are added directly to the Product Backlog** (not a sprint log).
  - The Product Backlog acts as the master, prioritized inventory of all desired features, enhancements, and requirements for the product.

- **Managed & Prioritized by the Product Owner (PO):**
  - The **Product Owner** owns, maintains, and continuously prioritizes the Product Backlog based on business value, user needs, and stakeholder feedback. High-value stories are placed at the top, while complex or low-priority stories (like *epics*) stay at the bottom.
- **Refinement (Backlog Grooming):**
  - Before stories are pulled into an upcoming development cycle, the team goes through **Backlog Refinement**. During these sessions, developers, testers, and the Product Owner discuss the stories to:
    - Clarify ambiguities.
    - Break down large items (*epics*) into smaller, independent stories.
    - Estimate the effort required (sizing/story points).
    - Define the **Acceptance Tests** (Confirmation) so everyone agrees on what "done" means.
- **Moved to the Sprint Backlog (Sprint Log):**
  - When a new sprint planning meeting begins, top-priority, well-refined stories are pulled from the *Product Backlog* and moved into the **Sprint Backlog** (Sprint Log) for the upcoming iteration (e.g., a 2-week sprint). Only then are they actively worked on, designed, coded, and tested by the development team.

📝 EXAMPLE 4: **Case Study: Backlog Grooming and Task Breakdown for Book Search**

Phase 1: Initial Elicitation & Adding to the Product Backlog

> **User Story Card (Draft):**
>
> - **As a** student,

> - **I want** to search for books,
> - **So that** I can quickly find the materials I need for my courses.

- **Where it goes:** This rough, high-level story is added directly to the **Product Backlog** as a prioritized item, but it is currently too broad (an *epic* or unrefined story) to be coded immediately.

Phase 2: Product Backlog Refinement (Backlog Grooming)

Before a story can be pulled into a sprint, the development team, QA engineers, and the Product Owner hold a **Refinement Session**. During this meeting, they inspect the story against the **INVEST** criteria (ensuring it is independent, negotiable, valuable, estimable, small, and testable).

1. **Identifying Ambiguities (The Problem with Broad Stories):**
   - *Developer asks:* "What search criteria should be supported? Just title, or author and ISBN too? And how many results should show per page?"
2. **Refining and Splitting:**
   - The Product Owner clarifies the exact scope for the upcoming release.
   - They define the explicit **Acceptance Tests (Confirmation)** right on the back of the card (Section 3.3.1):
     - *Acceptance Test 1:* Search for books using an ISBN.
     - *Acceptance Test 2:* Search for books using an author's name.
     - *Acceptance Test 3:* Search for books using a title.
     - *Acceptance Test 4:* Search for books added on or after a specific date.

3. **Estimation:**
    - The team assigns a size/effort score (e.g.,*3 Story Points*) because the refined criteria make the work clear and small enough to complete within a sprint.

Phase 3: Sprint Planning & Moving to the Sprint Backlog

During the Sprint Planning meeting for **Sprint 1**, the team reviews the top of the Product Backlog.

- Because the book-search story has now been fully **refined** and estimated, the Product Owner pulls it from the master **Product Backlog** and places it into the **Sprint Backlog** (Sprint 1).

Phase 4: Execution on the Scrum Board (Task Breakdown)

Once inside the sprint, the development team takes the refined user story and breaks it down into technical implementation tasks on their **Scrum Board**:

- **Tasks Created for this Story:**
    1. *Database Task:* Add indexes for ISBN, title, and author fields in PostgreSQL.
    2. *Backend Task:* Implement REST API endpoint `GET /api/books/search?query=...` with filtering logic.
    3. *Frontend Task:* Design and code the search input bar and results grid component.
    4. *QA Task:* Write automated unit and integration tests covering Acceptance Tests 1–4.

As the team executes the work, the story flows across the Scrum board columns:

Phase 5: Review and Acceptance (Confirmation)

- Once all technical tasks are finished, the story moves to **Testing / Review**.
- The QA engineer or Product Owner runs through the pre-defined **Acceptance Tests (Confirmation)** specified during the refinement phase (e.g., typing an ISBN to ensure the correct book appears).
- Because all criteria pass successfully without "gold plating" (unauthorized extra features), the user story is officially marked as **Done** and is ready to be deployed in the working software increment at the end of the sprint!

7 Use Cases

In plan-driven or Waterfall software development, requirements engineering happens upfront in a dedicated, comprehensive phase before design and coding begin. Because projects following this methodology rely on detailed, stable documentation to mitigate costly rework, lightweight tools like agile user stories are often insufficient.

Use Cases bridge this gap, they serve as a formal, highly detailed textual specification of system behavior. They act as a clear, written agreement between stakeholders and the development team, ensuring everyone shares the exact same vision of what the system must achieve before engineering resources are committed.

📖 DEFINITION 3: **Use Case**

A **use case** is a structured textual document that specifies the functional requirements of a system from an external **actor's** perspective.

📖 DEFINITION 4: **The Actor**

An entity external to the system interacting with it to achieve a specific goal. Typically a human user, but it can also be another software system or external hardware component.

ⓘ **Note**

Use cases are written from the perspective of an **actor** interacting with the system to achieve specific objectives. Typically, the actor is a human user, although it can also be another software or hardware component. In all cases, the actor is an entity external to the system.

A use case breaks an operation down into two primary lists of steps:

1. **Main Flow (Happy Path):** The sequence of steps executed when everything goes successfully without errors or interruptions.
2. **Extensions:** Alternative paths, specialized variations, or error-handling steps triggered at specific points in the main flow.

WRITING EFFECTIVE USE CASES

Alistair Cockburn defines a use case as a contract for behavior that captures the interactions between a system and external actors to achieve a specific user goal. Rather than listing internal UI widgets or isolated technical specs, effective use cases focus on stakeholder value and behavioral requirements

OPEN PDF

7.1 The Formal Way to Write a Use Case

A well-structured, formal use case follows established industry guidelines (such as those by Alistair Cockburn) and typically incorporates the following standard format:

- **Title:** Must start with an active verb in the base form (e.g.,*Transfer Values between Accounts*).
- **Primary Actor:** The external entity initiating the use case.
- **Pre-conditions:** Conditions that *must* be true before the use case can start (e.g.,*Customer must be logged into the banking portal*).
- **Post-conditions:** State of the system after successful execution (e.g.,*Funds are debited from source and credited to destination account*).
- **Main Flow (Happy Path):** Numbered steps written in simple language, highlighting active actions taken by the actor and responses by the system. Included use cases are often underlined (e.g., `1 - Authenticate Customer`).
- **Extensions:** Alphabetical/numeric modifiers mapping back to specific main flow steps to handle alternative scenarios or errors (e.g., `3a - If transfer amount exceeds current balance, system requests new amount`).

📝 EXAMPLE 5: **Transfer Values between Accounts**

Transfer Values between Accounts

- **Actor:** Bank Customer
- **Pre-conditions:** The customer is authenticated and has an active account balance.
- **Main Flow:**
  1. Authenticate Customer
  2. Customer enters destination account and branch.
  3. Customer enters the amount for transfer.
  4. Customer specifies the transfer date.

5. System executes the transfer.
6. System asks if the customer wants to make another transfer.

- **Extensions:**
  - `2a` - If the account or branch is incorrect, system requests a new account and branch.
  - `3a` - If transfer amount exceeds current balance, system requests a new amount.
  - `5a` - If the transfer date is the current date, system processes the transfer immediately.

7.1.1 The core rules and good practices for writing effective use cases:

The core rules and good practices for writing effective use cases:

1. Structural and Syntax Rules

- **Name with a Verb:** Every use case name must start with a verb in the base form (e.g.,*Transfer Values between Accounts*).
- **Identify the Main Actor:** Clearly specify the external actor interacting with the system.
- **Include Other Use Cases:** You can include another use case within a flow. The syntax requires underlining the included use case's name (e.g., Authenticate Customer), meaning all its steps must execute before proceeding (similar to a macro).
- **Utilize Extensions Effectively:** Extensions serve two main purposes:
  1. Detailing a step in the main flow (e.g., branching behavior based on dates).
  2. Handling errors, exceptions, and cancellations (e.g., insufficient balance).

- **Avoid Decision Statements in Main Flows:** Avoid programming-like conditional statements (such as `if` or `repeat until`) in the main flow. Instead, branch normal alternatives and error checks out into **extensions**.

2. Recommended Sections

In addition to the main flow and extensions, use case descriptions can include:

- **Purpose:** What the use case achieves.
- **Pre-conditions:** Conditions that must be true *before* execution.
- **Post-conditions:** State of the system *after* successful execution.
- **Related Use Cases:** Links to dependent or included operations.

3. Good Practices for Writing Use Cases

- **Use Simple, Accessible Language:** Write as if explaining it to an early elementary school student. Each step should clearly state an active verb performed by the actor or the system (e.g.,*The customer inserts the card into the ATM*,*The system validates the inserted card*).
- **Keep It Concise:** Following Alistair Cockburn's recommendation (`[link](https://dl.acm.org/citation.cfm?id=517669)`), keep the **main flow to a maximum of nine steps**. If a use case gets too complex, break it down or group related steps together (e.g., combining *User enters login* and *User enters password* into *User enters login and password*).
- **Maintain a High Level of Abstraction:** Use cases are requirements documents,**not algorithms or pseudo-code**. Avoid procedural loops; instead, use descriptive natural language phrases like *"The customer browses the catalog until finding the desired product."*
- **Omit Technology and UI Details:** Do not mention specific user interfaces, button colors, hardware choices, or design aspects (e.g., avoid writing *"The customer presses the green button"*). Focus strictly on *what* the system should do, not *how* it will be implemented.

- **Avoid Trivial CRUD Use Cases:** Do not create separate, trivial use cases for every individual database operation like *Create Student*,*Retrieve Student*, etc. Instead, group them under a single comprehensive use case like *Manage Student*.
- **Enforce Consistent Vocabulary:** Use the exact same terms across all use cases (avoid mixing words like *Customer* and *User* interchangeably). It is recommended to maintain a project **glossary** as suggested in *The Pragmatic Programmer* to ensure common understanding between users and developers.

7.1.2 Examples

📝 EXAMPLE 6: **Use Cases for a Library Management System**

Review the following high-level requirements for a Library Management System:

- **FR-01:** The system must allow authenticated students and instructors to search for books by ISBN, author, title, or publication date.
- **FR-02:** The system must allow students to borrow available books from the library.
- **FR-03:** The system must allow students to return borrowed books.
- **FR-04:** The system must allow students to renew active book loans.
- **FR-05:** The system must allow students to reserve books that are currently out on loan.
- **FR-06:** The system must allow instructors to request/recommend new books for library acquisition.
- **FR-07:** The system must allow library staff members to register new users in the system.
- **FR-08:** The system must allow library staff members to add new books or remove damaged books from the collection.

✅ Show Solution

Use Case 1: Search for Books

Use Case 1: Search for Books

- **Mapped Requirements:** FR-01
- **Primary Actor:** Library User (Student or Instructor)
- **Pre-conditions:** The user has access to the library catalog interface.
- **Main Flow:**
  1. User selects search criteria (ISBN, author, title, or date).
  2. User enters search keywords or parameters.
  3. System validates the search parameters.
  4. System queries the book catalog and displays matching results.
- **Extensions:**
  - `3a` - If search parameters are empty or invalid, system requests valid input.
  - `4a` - If no matching books are found, system displays a "No results found" message and suggests alternative keywords.

Use Case 2: Borrow Book

- **Mapped Requirements:** FR-02
- **Primary Actor:** Student
- **Pre-conditions:** Student is authenticated and has no active borrowing suspensions or overdue fines.
- **Main Flow:**
  1. Search for Books
  2. Student selects an available book from the search results.
  3. Student requests to borrow the selected book.

4. System verifies that the book is available and the student's account is in good standing.
5. System records the loan, updates the book status to on-loan, and generates a due date.

- **Extensions:**
  - `1a` - If the student is not logged in, system triggers Authenticate Customer.
  - `4a` - If the book is currently unavailable, system informs the student and offers the option to reserve it via Reserve Book.
  - `4b` - If the student has reached their maximum borrowing limit or has overdue fines, system rejects the loan and displays an error message.

Use Case 3: Reserve Book

Use Case 3: Reserve Book

- **Mapped Requirements:** FR-05
- **Primary Actor:** Student
- **Pre-conditions:** The desired book is currently out on loan to another user.
- **Main Flow:**
  1. Search for Books
  2. Student selects an unavailable book from the catalog.
  3. Student requests a reservation for the book.
  4. System checks student reservation eligibility.
  5. System adds the student to the book's reservation queue and confirms the reservation.
- **Extensions:**
  - `4a` - If the student has already reserved or currently holds a copy of the book, system rejects the duplicate reservation request.

Use Case 4: Manage Book Inventory

Use Case 4: Manage Book Inventory

- **Mapped Requirements:** FR-08
- **Primary Actor:** Library Staff Member
- **Pre-conditions:** Staff member is authenticated with administrative privileges.
- **Main Flow:**
  1. Staff member selects an inventory action (add new book or remove damaged book).
  2. Staff member inputs book details or scans book identification codes.
  3. System validates the provided book data.
  4. System updates the catalog database to reflect the inventory change.
- **Extensions:**
  - `3a` - If mandatory book details are missing or formatted incorrectly, system requests correct information.
  - `4a` - If a book marked for removal has active loans or pending reservations, system displays a warning and requires confirmation before archiving.

📝 EXAMPLE 7: **Multiple detailed use cases**

Functional Requirement

- **FR-01:** The system must allow users to manage their product orders in the online store.

(Notice how broad this requirement is—it covers placing, modifying, canceling, and tracking orders. Trying to force all of this into a single use case would violate Alistair Cockburn's rule of keeping the main flow concise and under 9 steps. Instead, we break it down into multiple distinct use cases.)

✅ Show Solution

Corresponding Use Cases (Multiple)

Use Case A: Place New Order

- **Primary Actor:** Online Store Customer
- **Pre-conditions:** Customer is authenticated, has items in the shopping cart, and has a saved delivery address.
- **Main Flow:**
  1. Customer views the items in the shopping cart and initiates checkout.
  2. Customer confirms the delivery address and shipping method.
  3. Customer selects the payment method.
  4. Process Payment
  5. System generates an order confirmation number and sends a confirmation receipt.
- **Extensions:**
  - `4a` - If payment authorization fails, system prompts the customer for a different payment method.

Use Case B: Cancel Pending Order

- **Primary Actor:** Online Store Customer
- **Pre-conditions:** Customer is authenticated and has an active order that has not yet entered the shipping phase.
- **Main Flow:**
  1. Customer views their order history.
  2. Customer selects the specific pending order to cancel.
  3. Customer requests to cancel the order and provides a cancellation reason.
  4. System verifies that the order status permits cancellation.
  5. System cancels the order, restores inventory levels, and initiates a refund if applicable.

- **Extensions:**
  - `4a` - If the order has already been shipped or processed for delivery, system rejects the cancellation request and displays an explanatory message.

Use Case C: Track Order Status

- **Primary Actor:** Online Store Customer
- **Pre-conditions:** Customer has at least one placed order.
- **Main Flow:**
  1. Customer opens their order tracking dashboard.
  2. Customer selects an order from the list.
  3. System retrieves the latest shipping and fulfillment status from the logistics partner.
  4. System displays the current tracking timeline and estimated delivery date to the customer.
- **Extensions:**
  - `3a` - If the tracking service connection fails, system displays the last known cached status and a "Data temporarily unavailable" warning.

📝 EXAMPLE 8:

Multiple use cases derived from a single functional requirement

You are specifying requirements for an online banking platform using plan-driven development. Consider the following high-level functional requirement:

- **FR-01:***The system must allow authenticated customers to manage their bank accounts, including viewing account statements and transferring funds to other accounts.*

Questions:

1. Explain why a broad functional requirement like **FR-01** should be broken down into **more than one use case.**

2. Write **two distinct formal use cases** to fulfill **FR-01**:

- **Use Case A:***View Account Statements*
- **Use Case B:***Transfer Values between Accounts*

Ensure both use cases adhere to all structural rules:

- Use verb-first base names.
- Keep the main flow under 9 steps.
- Avoid procedural conditional statements (`if` / `else`) in the main flows, placing error handling and alternatives into the **extensions** section.
- Properly include the security requirement by utilizing the `<u>Authenticate Customer</u>` included use case with correct underlining syntax in Step 1 of both flows.

✅ Show Solution

1. Why FR-01 Expands into Multiple Use Cases

Attempting to cram viewing statements, fund transfers, and other account management tasks into a single use case would result in an overly complex, lengthy document that violates this principle and obscures specific user goals. Therefore, broad functional requirements must be decomposed into distinct, focused operational units.

2. Formal Use Cases

Use Case A: View Account Statements

- **Primary Actor:** Bank Customer
- **Pre-conditions:** The system is operational and connected to the account ledger database.
- **Main Flow:**
  1. Authenticate Customer
  2. Customer selects the account statement option from the main menu.
  3. Customer specifies the desired billing month and year.

4. System queries the database and retrieves the statement records.
5. System displays the formatted account statement to the customer.

- **Extensions:**
  - `1a` - If authentication fails or credentials are invalid, system displays an error message and terminates the use case.
  - `1b` - If the customer has not logged in during the active session, system triggers Authenticate Customer automatically.
  - `4a` - If no statement records exist for the selected period, system displays a "No records found" notification.

Use Case B: Transfer Values between Accounts

- **Primary Actor:** Bank Customer
- **Pre-conditions:** The customer is authenticated and has an active account balance.
- **Main Flow:**
  1. Authenticate Customer
  2. Customer enters destination account and branch.
  3. Customer enters the amount for transfer.
  4. Customer specifies the transfer date.
  5. System executes the transfer.
  6. System asks if the customer wants to make another transfer.
- **Extensions:**
  - `1a` - If authentication fails, system terminates the use case.
  - `2a` - If the account or branch is incorrect, system requests a new account and branch.
  - `3a` - If transfer amount exceeds current balance, system requests a new amount.
  - `4a` - Transfer date must be the current date or within one year in the future.
  - `5a` - If the transfer date is the current date, system processes the transfer immediately.
  - `5b` - If the transfer date is in the future, system schedules the transfer.

8 Use Case Diagrams

While the core substance of requirements modeling lies in textual specifications, the Unified Modeling Language (UML) introduces the **Use Case Diagram** to provide a high-level visual perspective of a system.

📖 DEFINITION 5: **Use Case Diagram**

A **Use Case Diagram** serves as a graphical "visual catalog" or architectural blueprint of a system. It maps out:

- **System Boundary:** Represented as a rectangle enclosing the system features, separating internal software operations from external entities.
- **Actors:** Represented as stick figures positioned *outside* the system boundary, illustrating the human users, hardware, or external systems that interact with the software (e.g.,*Customer*,*Manager*).
- **Use Cases:** Represented as horizontal ellipses inside the boundary box, depicting the primary functional goals or features the system provides.
- **Relationships:** Connecting lines that show participation between actors and use cases, or specialized structural relationships such as `<<includes>>` (as seen in the banking system where *Transfer Funds* includes *Authenticate Customer*).

8.1 Use Case Diagrams vs. Textual Use Cases

- **Use Case Diagrams (The Visual Map):** You typically maintain **one master diagram** (or a few high-level architectural ones) for the entire system. It provides an executive overview of who interacts with the system and what high-level features exist, but it contains **zero step-by-step logic**.

- **Textual Use Cases (The Detailed Chapters):** You write a separate, rigorous text document for *each distinct user goal or feature* (e.g.,*View Account Statements*,*Transfer Values between Accounts*). If a system has 20 functional goals, you maintain 20 distinct textual documents.

> ⓘ **Use Cases vs. Use Case Diagrams**
>
> - **Textual Use Cases (1-to-1 Mapping):** If your system has 20 distinct functional requirements or user goals, you will write **20 separate textual use case documents**, each detailing its own pre-conditions, main success flows, and extension paths.
> - **Use Case Diagrams (Many-to-1 Mapping):** You typically maintain **only one master visual diagram** (or a few high-level architectural ones) enclosed within a system boundary box. This single UML diagram acts as a catalog, mapping out all actors and use cases together in a single bird's-eye view.

8.1.1 When to Use Each Approach

In a rigorous software engineering process, textual documents and visual diagrams complement each other rather than compete. You use them at different stages and for different purposes:

- **When to Use Use Case Diagrams:**
  - **At Project Inception:** To scope the project, identify key system boundaries, and map out major stakeholder interactions during early planning.
  - **For Executive Overviews:** When presenting the system architecture to high-level stakeholders or project sponsors who need a bird's-eye view of functional modules without getting bogged down in procedural steps.
- **When to Use Textual Use Cases:**
  - **During Detailed Requirements Specification:** Whenever developers and customers need to reach a precise, binding agreement on exactly how a feature

must behave

- **During Development and Testing:** When engineers need to implement exact operational workflows, handle error states via extensions (`2a` , `3a`), and write acceptance test cases.

8.2 Use Case Diagram objects

At a high level, a Use Case Diagram consists of four fundamental building blocks, as outlined by standard Unified Modeling Language (UML) specifications:

1. **Actor**: Actor in a use case diagram is any entity that performs a role in one given system. This could be a person, organization or an external system and usually drawn. Actors are strictly external to the system boundary and can be categorized as *primary* (users who achieve a direct goal),*supporting* (systems/services that provide assistance), or *offstage* (stakeholders interested in outcomes but not directly executing actions).

2. **Use Case:** A specific function, service, or task that the system performs to yield a measurable value or goal for an actor. Drawn as a horizontal ellipse and typically named using a verb-noun phrase (e.g.,*Check Account Balance*,*Place Order*).

3. **System Boundary:** A rectangular boundary box that delineates the scope of the software system being developed.It clearly separates what falls inside the automated system capabilities from what remains outside in the external environment (where actors reside).

4. **Package:** An optional grouping mechanism used in complex, enterprise-scale projects to organize related use cases into manageable subsystems or modules.

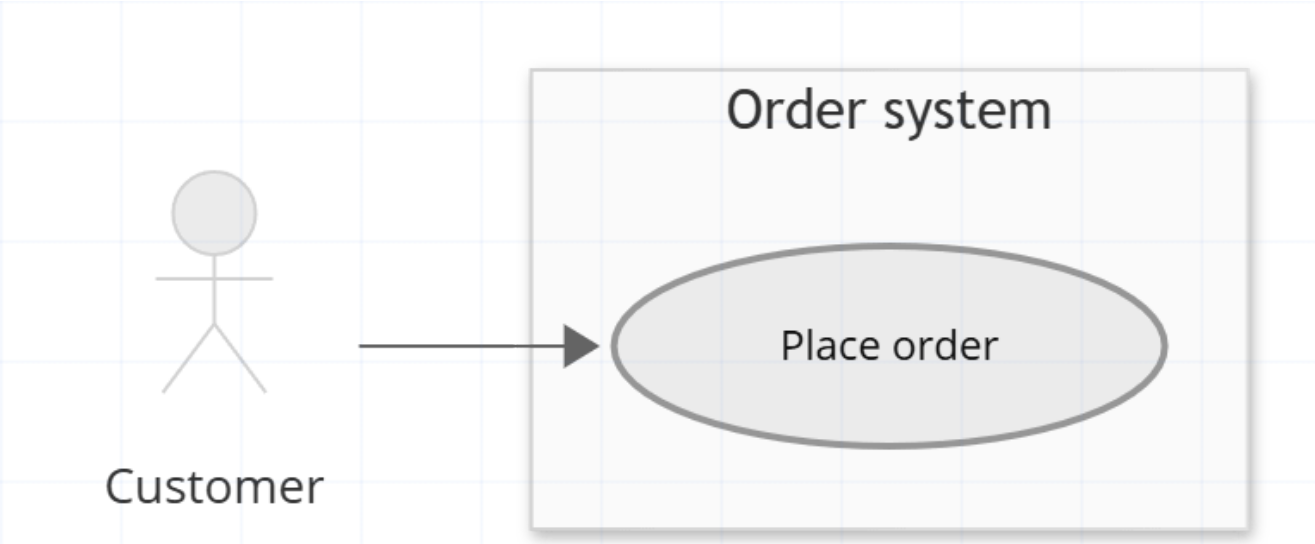


Diagram 1: Use Case Diagram Components

EXAMPLE 9:

A web application allows a user to:

- read the products in the catalog
- search products in the catalog
- buy a product

Make a use case diagram to model this system.

✅ Show Solution

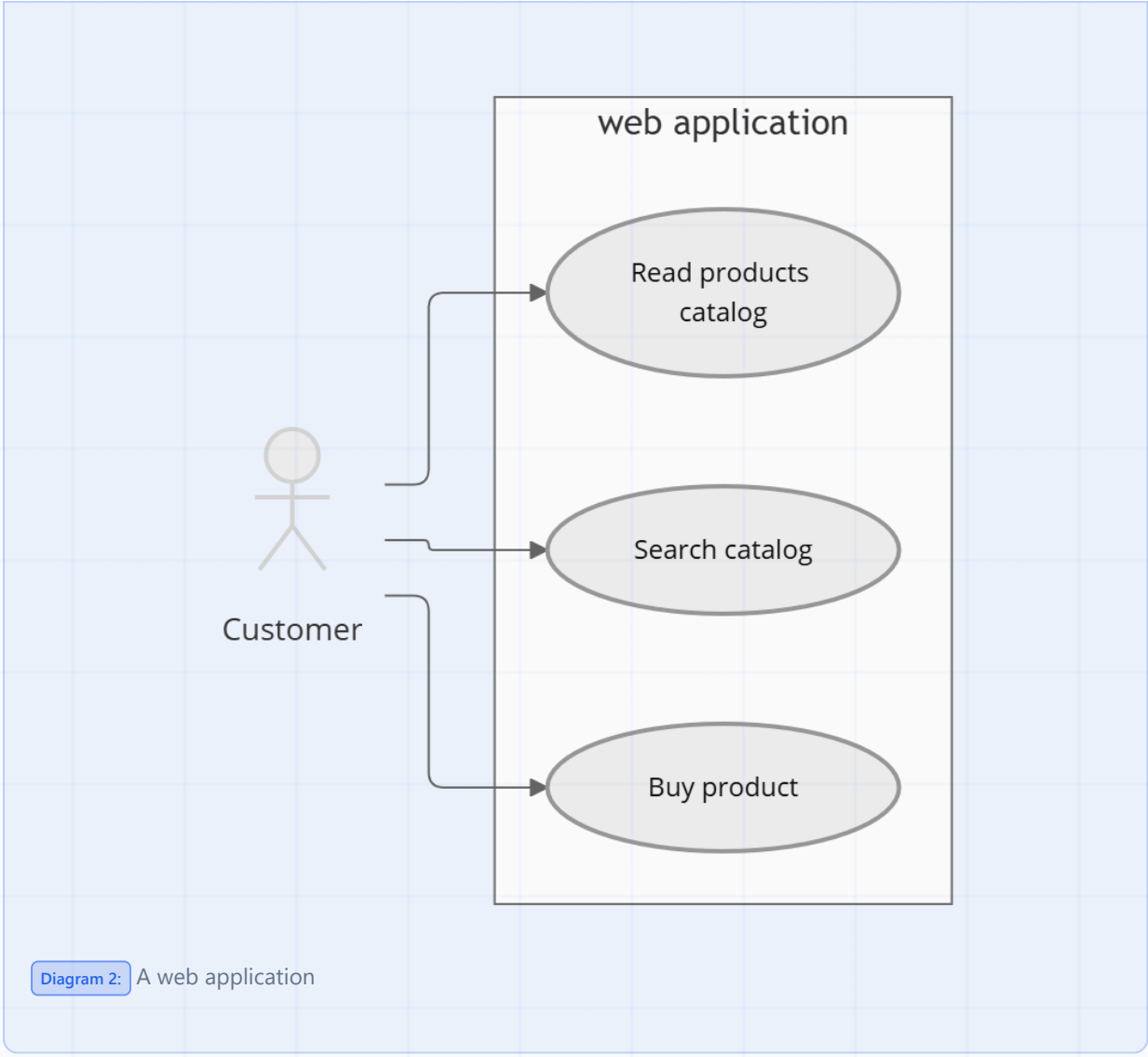


Diagram 2: A web application

8.2.1 Use Case Diagram Relationships

In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases.

8.2.1.1 Association Relationship

Association Relationship shows the interaction between an actor and a use case. It is represented by a line connecting the actor and the use case.

📝 EXAMPLE 10:

- **Actor:** Customer
- **Use Case:** Transfer Funds

- **Association:** A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.

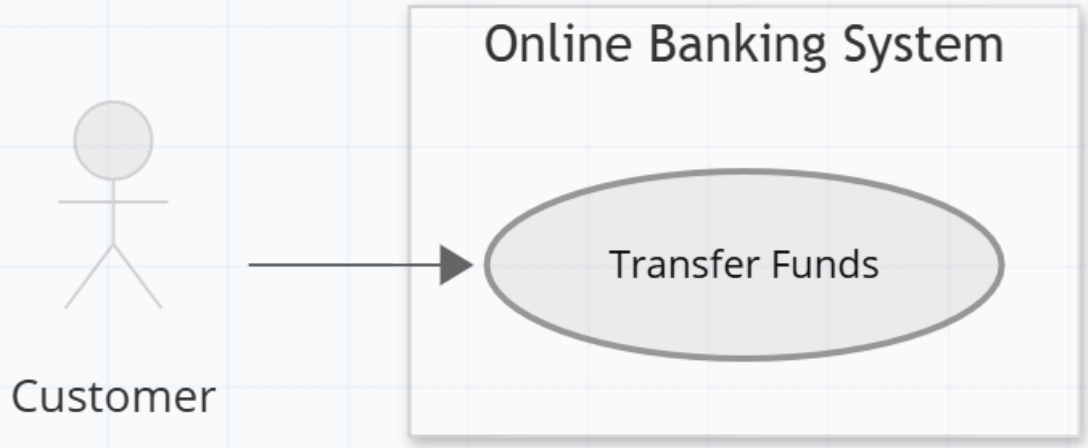


Diagram 3: Transfer Funds

8.2.1.2 Include Relationship

An **include** relationship (sometimes called *sub-use case* or *uses*) specifies that the behavior of a **base use case** explicitly incorporates all the behavior defined in an **included use case**.

- **When to use:** Use it to represent mandatory, shared behavior that occurs in multiple use cases, allowing you to factor out common steps and prevent duplication.
- **Execution:** The execution of the included use case is mandatory whenever the base use case runs. All steps of the included use case must be completed before the system proceeds.
- **Notation (UML):** Represented by a dashed arrow pointing from the base use case to the included use case, labeled with the keyword `<<include>>`.

📝 EXAMPLE 11:

A high-level overview of the banking system's primary workflows. It shows that Customers can withdraw money and transfer funds, while Managers can open

accounts. Both customer-facing financial transactions require mandatory customer authentication (`<<include>>`) to ensure system security.

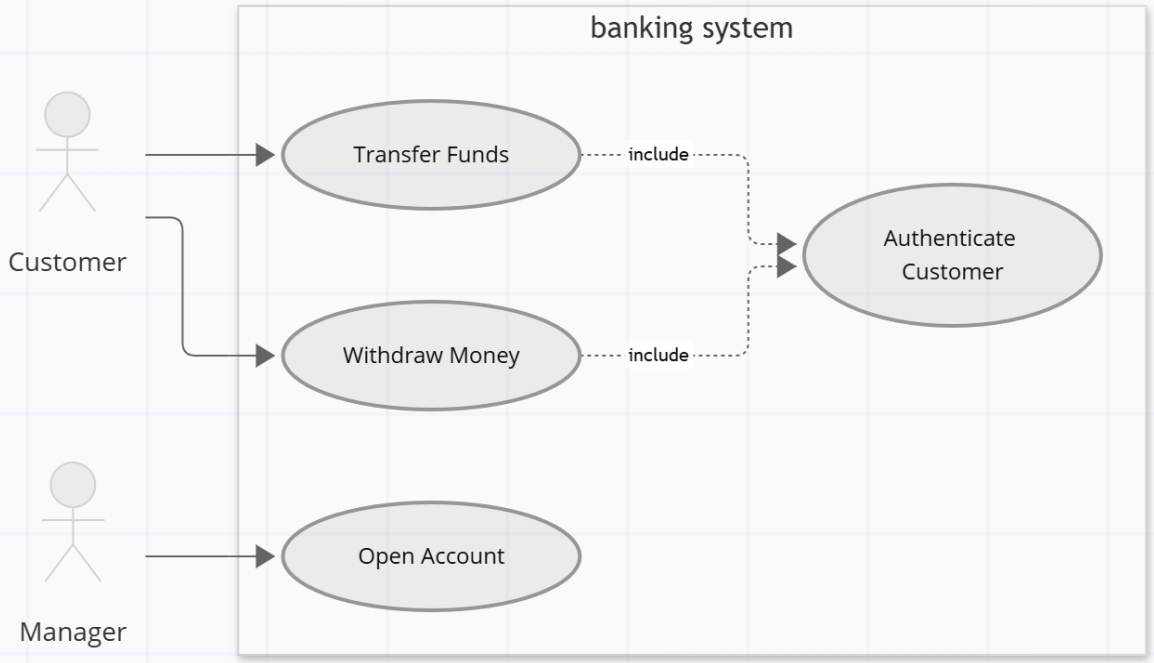


Diagram 4: Core Banking System Use Cases

8.2.1.3 The Extend Relationship

An **extend** relationship specifies that an **extension use case** may optionally augment or insert behavior into a **base use case** under certain conditions (such as handling errors, optional features, or specific alternate paths).

- **When to use:** Use it to model optional behavior, special-case scenarios, or exception handling that *does not* always happen, keeping the main "happy path" clean and uncluttered.
- **Execution:** The base use case can execute completely on its own without the extension ever running. The extension only triggers if a specific condition (an "extension point") is met.
- **Notation (UML):** Represented by a dashed arrow pointing *from* the extension use case *back to* the base use case, labeled with the keyword `<<extend>>` .

EXAMPLE 12:

This use case diagram models the checkout workflow of an e-commerce platform. The primary actor is the **Customer**, who initiates the base use case **"Checkout Order"**.

To ensure a successful transaction, the system mandatorily executes the **"Process Payment"** use case, modeled with an `<<include>>` relationship (as payment is a required step).

However, the diagram also features two `<<extend>>` relationships: **"Apply Promo Code"** and **"Request Gift Wrapping"**. Unlike `<<include>>`, an `<<extend>>` relationship represents *optional* behavior. These extended use cases only execute under specific conditions (e.g., if the customer actively chooses to enter a discount code or selects gift wrapping). They add functionality to the base "Checkout Order" use case without being mandatory for every transaction.

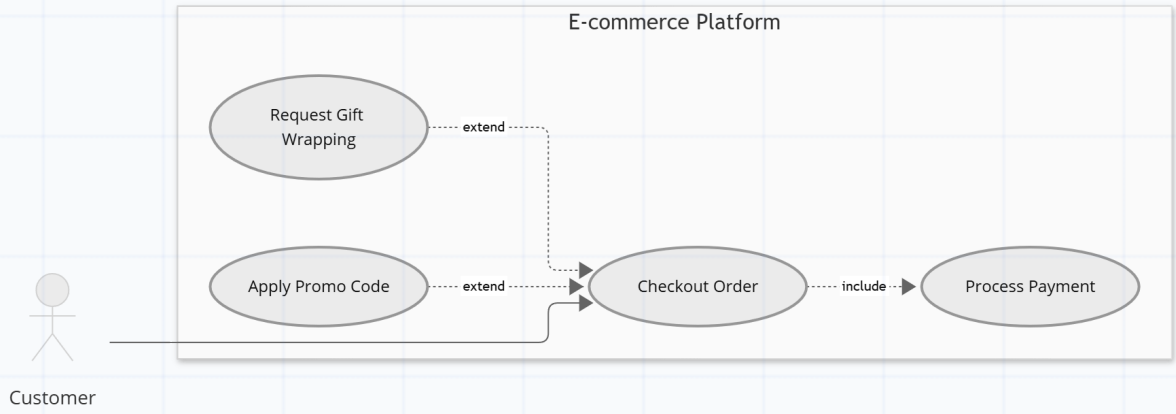


Diagram 5: E-Commerce Checkout Process Use Case Diagram

extend vs include

- `<<include>>` **(Mandatory)**: The base use case *cannot* complete without it. (e.g., You cannot checkout without processing payment). Syntax: `Base ..> : include RequiredStep`
- `<<extend>>` **(Optional)**: The extending use case adds extra behavior to the base use case *only if* certain conditions are met. (e.g., You can checkout perfectly fine without a promo code, but if you have one, this step extends the flow). Syntax: `OptionalStep ..> : extend Base`

8.2.1.4 Solved Examples

EXAMPLE 13: Online Quiz Management System — Use Case Model

A university is developing an Online Quiz Management System to facilitate automated testing and quality-controlled question authoring. The system involves three distinct types of users: **Students** (who take quizzes), **Tutors** (who author content), and **Examinators** (who validate content).

When a Student requests a quiz, the system automatically retrieves a set of questions from its database, composes them into a cohesive quiz, and presents them. After the Student submits their answers, the system automatically grades them. During the quiz, if a Student is stuck, they may optionally request a hint.

Functional Requirements

- **FR-1:** The system shall allow a Student to request and initiate a quiz session.
- **FR-2:** The system shall automatically select a predefined number of questions from the database to compose the quiz.
- **FR-3:** The system shall automatically evaluate and rate the Student's answers upon submission.

- **FR-4:** The system shall allow a Student to request a hint for the current question during an active quiz.
- **FR-5:** The system shall allow a Tutor to create and submit new questions to the question bank.
- **FR-6:** The system shall allow a Tutor to create and associate hints with existing questions.
- **FR-7:** The system shall allow an Examinator to review pending questions and mark them as "Certified" or "Rejected".

Business Rules

- **BR-1:** The system shall only select questions that have a "Certified" status when composing a quiz.
- **BR-2:** A question cannot be marked as "Certified" until an Examinator verifies it is not trivial and is logically sensical.
- **BR-3:** Hints are strictly optional; the quiz flow must not be interrupted or delayed if a Student does not request a hint.

Non-Functional Requirements

- **NFR-1 (Security):** The system shall enforce Role-Based Access Control (RBAC) to ensure Students, Tutors, and Examinators can only access their respective functionalities.
- **NFR-2 (Performance):** The system shall compose and load a 50-question quiz within 2 seconds of the Student's request.

To maintain quality, Tutors are responsible for writing new questions and hints. However, these cannot be used immediately. An Examinator must review and "certify" each question to ensure it is logically sound, unambiguous, and of appropriate difficulty. Only certified questions are eligible to be picked for a Student's quiz.

✅ Show Solution

Table 3: *Actor Definitions*

Actor	Description
User	A student or learner who takes quizzes to test their knowledge
Tutor	An educator who creates and provides questions and hints for the quiz database
Examinator	A quality assurance role responsible for reviewing and certifying questions before they can be used in quizzes

Table 4: *Use Case Relationships*

Relationship Type	Source Use Case	Target Use Case	Reason
Include	Take Quiz	Pick Questions from DB	Mandatory: Quiz cannot exist without questions
Include	Take Quiz	Compose Quiz	Mandatory: Questions must be assembled into a quiz format
Include	Take Quiz	Rate Answers	Mandatory: All submissions must be graded
Extend	Request Hint	Take Quiz	Optional: Hints only shown if user requests them

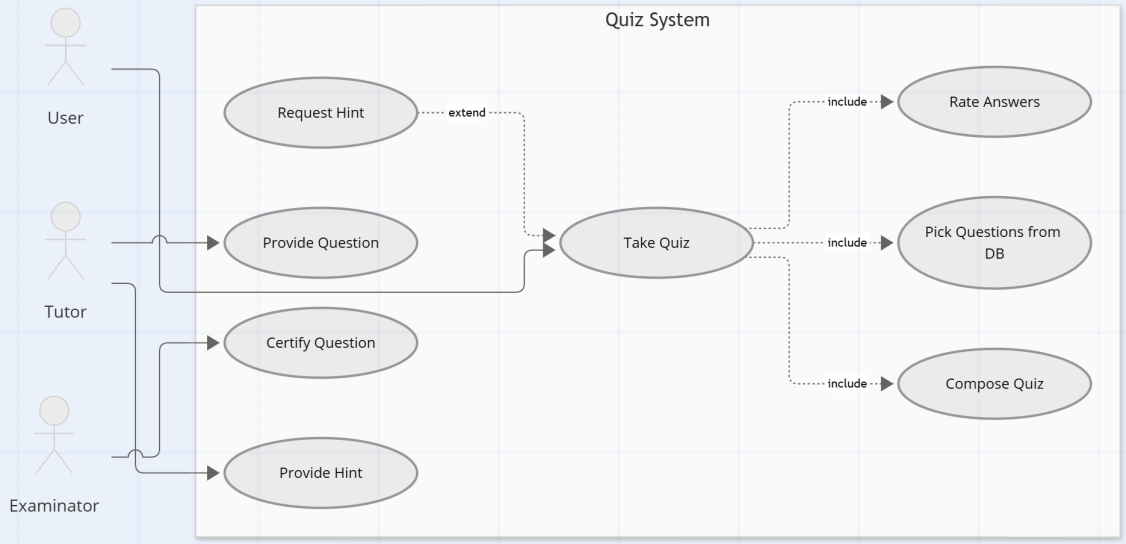


Diagram 6: Online Quiz Management System — Use Case Model